

Kubernetes in Action — 4장 실습 내역 요약

실습일자: 2026-04-26 · 실습환경: macOS (Apple Silicon) · Docker Desktop · kind v0.31.0 · k9s v0.50.18 · Kubernetes v1.35.0 · 클러스터 구성: control-plane × 1, worker × 2 (kind-ch4)

목차

- 0. 환경 세팅
- 4.1 Liveness Probe — 컨테이너 자가치유 시연
- 4.2 ReplicationController — 컨트롤러 reconciliation 4가지 시나리오
- 4.3 ReplicaSet — apiVersion/selector 차이
- 4.4 DaemonSet — 멀티노드 배치 시연
- 4.5 Job — 단일 / 병렬 (5번 완료)
- 4.6 CronJob — schedule + 수동 트리거 트릭
- 정리 + 회고

0. 환경 세팅

🎯 목적

4장 시연 전체에 필요한 환경을 준비한다. 책의 minikube 대신 **kind 멀티노드**를 선택한 이유는 DaemonSet 시연 시 노드별 Pod 분포를 시각적으로 보여주기 위함. **k9s**는 발표 중 **kubectl get** 반복 없이 리소스 변화를 실시간으로 보여주기 위함.

- 확인하려는 것: 컨트롤플레인 1 + 워커 2가 모두 **Ready**, **ch4** 네임스페이스가 기본값

설치 / 클러스터 생성

```
# 도구 설치
brew install kubectl kind k9s

# Docker Desktop 시작
open -a Docker

# 멀티노드 클러스터 (control-plane × 1, worker × 2)
kind create cluster --config kind-cluster.yaml

# 발표용 네임스페이스
kubectl create namespace ch4
kubectl config set-context --current --namespace=ch4
```

kind-cluster.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: ch4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

실행 결과

항목	값
Context	kind-ch4
Nodes	ch4-control-plane, ch4-worker, ch4-worker2 (모두 Ready)
Default namespace	ch4
대시보드	k9s (터미널 분할 — 좌: kubectl / 우: k9s)

4.1 Liveness Probe

🎯 목적 — 컨테이너 자가치유 확인

프로세스가 "살아있는데 응답 못 하는" 상태(hang/deadlock 등)일 때, K8s가 그것을 감지해 자동으로 재시작하는지 검증한다.

- 확인하려는 것 1: **RESTARTS** 카운터가 0 → 1 로 자동 증가 (kubelet이 재시작했다는 직접 증거)
- 확인하려는 것 2: **describe Events**에 "Liveness probe failed → Killing → Started" 흐름 기록
- 확인하려는 것 3: 재시작 후에도 **이전 컨테이너 로그가 보존되는지** (**---previous**)

시나리오: 일정 요청 후 일부러 500을 응답하는 이미지(**luksa/kubia-unhealthy**)를 배포해, kubelet이 컨테이너를 자동 재시작하는 것을 관찰.

실행

```
kubectl apply -f manifests/kubia-liveness-probe.yaml
```

k9s 관찰

1. **:pods** → **kubia-liveness** Pod 등장 → STATUS **ContainerCreating** → **Running**
2. 약 80초 후 첫 Liveness 실패. 약 3~4분 후 **RESTARTS = 1** (재시작 1회)
3. **d** 키로 describe — Events 섹션에서 다음 메시지 확인:
 - **Liveness probe failed: HTTP probe failed with statuscode: 500**
 - **Container kubia failed liveness probe, will be restarted**
 - **Killing container with id docker://<container-id>**

핵심 발견: 로그 내용(l vs p)은 비슷해 보이지만 — 같은 이미지가 매번 같은 코드를 실행하기 때문. 재시작의 진짜 증거는 **RESTARTS 컬럼 숫자**와 **describe Events**의 **Killing** / **Started** 시간차.

정리

```
kubectl delete pod kubia-liveness
```

4.2 ReplicationController

🎯 목적 — 컨트롤러 reconciliation 루프 확인

RC가 "현재 상태"와 "목표 상태(replicas=N)"의 차이를 어떻게 해소하는지 4가지 각도에서 검증한다. 이게 K8s 컨트롤러의 근본 동작 모델.

- **시나리오 1:** Pod 1개 강제 삭제 → RC가 즉시 새로 채우는지
- **시나리오 2:** Pod 라벨 변경 → RC 관리에서 빠진 후 RC가 빈자리를 새 Pod로 채우는지 (= 라벨 매칭 기반 관리)
- **시나리오 3:** **scale** 명령으로 replicas 증가 → 즉시 추가 Pod 생성하는지
- **시나리오 4:** RC 삭제 시 cascade 정책에 따라 Pod까지 삭제할지(**foreground**) 살릴지(**orphan**) 결정 가능한지

시나리오: RC가 replicas=3을 유지하도록 4가지 시나리오로 reconciliation 동작 확인.

준비

```
kubectl apply -f manifests/kubia-rc.yaml
# RC: DESIRED=3 CURRENT=3 READY=3
```

시나리오 1 Pod 1개 죽이기

```
kubectl delete $(kubectl get po -l app=kubia -o name | head -1)
```

관찰: 죽인 Pod이 **Terminating** 으로 사라짐. 거의 동시에 새 Pod이 **ContainerCreating** 으로 등장 (AGE=0s). 결과적으로 여전히 3개.

실수했던 명령어 (참고)

처음 시도 — `kubectl delete pod $(kubectl get po -l app=kubia -o name | head -1)` 는 오류:

```
error: there is no need to specify a resource type as a separate argument
when passing arguments in resource/name form
```

이유: `kubectl get -o name` 은 `pod/kubia-XXXXX` 형태로 반환 → `delete pod pod/...` 로 중복.

수정: `kubectl delete $(...)` ← `pod` 단어 제거.

시나리오 2 라벨 변경 (RC 관리에서 빼기)

```
POD=$(kubectl get po -l app=kubia -o jsonpath='{.items[0].metadata.name}')
kubectl label pod $POD app=foo --overwrite
kubectl get po --show-labels
```

관찰: 총 Pod이 4개로 늘어남.

- 라벨 바꾼 Pod (`app=foo`) → RC 관리 대상에서 빠짐
- RC: "내 라벨(`app=kubia`) Pod이 2개네? 부족!" → 즉시 1개 추가
- 최종: `app=kubia` 3개 + `app=foo` 1개 (외톨이)

시나리오 3 스케일 업

```
kubectl scale rc kubia --replicas=5
```

관찰: `app=kubia` Pod이 3개 → 5개로 증가. `app=foo` 는 그대로 1개. 총 6개.

시나리오 4 RC 삭제 (cascade)

```
# 책: kubectl delete rc kubia --cascade=false (deprecated)
# 최신: kubectl delete rc kubia --cascade=orphan (RC만 삭제, Pod 살림)
# 또는: kubectl delete rc kubia (RC + Pod 모두 삭제)
kubectl delete rc kubia
```

실수담: 시나리오 4의 `--cascade=orphan` 명령이 실행 안 된 채로 진행됨 → "Pod을 죽여도 5개가 다시 살아남"는 의도치 않은 보너스 시연. 결국 **RC가 일하고 있다는 또 다른 증거.**

4.3 ReplicaSet

🎯 목적 — RC와의 차이점 확인

RC의 후계자인 ReplicaSet이 무엇이 달라졌는지 YAML 레벨에서 직접 확인한다. (동작 자체는 RC와 동일하므로 동작 시연보다 스펙 차이에 집중.)

- 확인하려는 것 1: `apiVersion` 이 `apps/v1` (책의 deprecated된 `extensions/v1beta1` 아님)
- 확인하려는 것 2: selector가 `matchLabels` + `matchExpressions` 구조 (RC의 평면 selector와 다름)
- 확인하려는 것 3: 실전에서는 RS를 직접 만들지 않고 Deployment가 자동 생성한다는 점

실행

```
kubectl apply -f manifests/kubia-replicaset.yaml
kubectl get rs
```

k9s에서 RC와 차이 확인

- `:rs` → `kubia` 행에서 `y` 키 → YAML 보기
- `apiVersion: apps/v1` (책의 `extensions/v1beta1` 이 아님)
- `selector.matchLabels` 구조 (RC의 평면 selector와 차이)

정리

```
kubectl delete rs kubia
```

요약: 동작은 RC와 거의 동일. 차이는 (1) `apps/v1` apiVersion, (2) `matchLabels` + `matchExpressions` 셀렉터. 실전에서는 ReplicaSet을 직접 안 만들고 **Deployment가 자동 생성하는** 형태로 사용.

4.4 DaemonSet

🎯 목적 — 노드 단위 추상화 확인

RS는 "Pod N개"인데, DaemonSet은 "노드 1개당 Pod 1개". 이 본질적 차이를 멀티노드 환경에서 시각적으로 검증한다.

- 확인하려는 것 1: `nodeSelector` 로 매칭되는 노드만 골라서 배포
- 확인하려는 것 2: 노드에 라벨을 동적으로 추가하면 즉시 그 노드에 Pod이 새로 생성
- 확인하려는 것 3: 라벨 제거하면 즉시 그 노드의 Pod이 회수
- 확인하려는 것 4: 매칭 노드 N개 = Pod N개 (`replicas` 명시 없이 자동)

시나리오: `disk=ssd` 라벨이 붙은 노드에만 모니터링 에이전트를 1개씩 실행. kind 멀티노드의 진가가 발휘되는 부분.

1단계 DaemonSet 생성 (라벨 붙은 노드 없음)

```
kubectl apply -f manifests/ssd-monitor-daemonset.yaml
kubectl get ds
# DESIRED=0 CURRENT=0 (라벨 매칭 노드 없음)
```

2단계 워커1번에 라벨

```
kubectl label node ch4-worker disk=ssd
```

관찰:

- `:ds` → `DESIRED=1 CURRENT=1 READY=1`
- `:pods` → `ssd-monitor-XXXXX` Pod 1개, NODE = `ch4-worker`

3단계 워커2번도 라벨 (멀티노드 효과)

```
kubectl label node ch4-worker2 disk=ssd
```

관찰: Pod이 2개로 자동 확장. NODE 컬럼이 각각 `ch4-worker`, `ch4-worker2`. "노드당 정확히 1개"가 시각적으로 보임.

4단계 라벨 제거

```
kubectl label node ch4-worker disk-
```

관찰: `ch4-worker` 의 Pod이 즉시 `Terminating` → 사라짐. `:ds` 에서 `DESIRED=1 CURRENT=1`.

정리

```
kubectl label node ch4-worker2 disk-  
kubectl delete ds ssd-monitor
```

핵심: DaemonSet은 "노드 단위 추상화". RS는 Pod 단위로 N개를 유지하지만, DaemonSet은 매칭되는 노드마다 자동으로 1개씩.

4.5 Job

🎯 목적 — 일회성 배치 워크로드 확인

RC/RS가 "항상 떠 있어야 하는 서비스"라면, Job은 "끝나야 하는 작업". 이 의미적 차이가 동작에 어떻게 반영되는지 검증한다.

- 확인하려는 것 1: 작업 완료 후 Pod이 다시 만들어지지 않고 **Completed** 로 보존되는지 (RC와의 결정적 차이)
- 확인하려는 것 2: Completed 상태에서도 로그/결과를 사후 조회 가능한지 (`kubectl logs`)
- 확인하려는 것 3: **parallelism** 으로 동시 실행, **completions** 로 총 횟수 제어

단일 Job

```
kubectl apply -f manifests/batch-job.yaml
```

관찰:

- Pod이 `Running` → 약 2분 후 → `Completed`
- `:jobs` 에서 `COMPLETIONS=0/1` → `1/1`
- Pod은 사라지지 않고 **Completed** 상태로 보존 (RC/RS와 결정적 차이)
- `kubectl logs -l app=batch-job` 로 결과 보존 확인

병렬 Job (completions=5, parallelism=2)

```
kubectl apply -f manifests/multi-completion-batch-job.yaml
```

관찰:

- 처음 2개 Pod 동시 Running (parallelism=2)
- 끝나면 다음 2개, 마지막 1개 → 총 5개 완료
- `:jobs` 에서 `COMPLETIONS=5/5`

정리

```
kubectl delete job batch-job multi-completion-batch-job
```

4.6 CronJob

🎯 목적 — 주기적 배치 워크로드 확인

CronJob은 "Job을 cron 일정대로 자동 생성하는 상위 객체". 단발성이 아닌 주기적인 ETL/리포트/정기 작업을 K8s 가 어떻게 표현하는지 검증한다.

- 확인하려는 것 1: `schedule` 표현식이 등록되고 시각이 되면 Job이 자동 생성되는지 (15분 못 기다리니 수동 트리거로 갈음)
- 확인하려는 것 2: `kubectl create job --from=cronjob/...` 로 즉시 한 번 실행 가능한지 (실전 디버깅 트릭)
- 확인하려는 것 3: `apiVersion` 이 반드시 `batch/v1` 인지 (책 1판의 `batch/v1beta1` 은 1.25에서 제거 됨)

1단계 — CronJob 생성

```
kubectl apply -f manifests/cronjob.yaml
kubectl get cj
```

관찰: `SCHEDULE=0,15,30,45 * * * *` (15분마다), `LAST SCHEDULE=<none>`

2단계 — 수동 트리거 (책에 없는 최신 트릭)

```
kubectl create job --from=cronjob/batch-job-every-fifteen-minutes manual-run-1
```

관찰: `manual-run-1` Job 즉시 생성 → Pod `Running` → `Completed`. 발표 시간에 15분 못 기다릴 때 유용.

apiVersion 주의: 책은 `batch/v1beta1`. 최신은 반드시 `batch/v1`. (Kubernetes 1.21에서 안정화, 1.25부터 `v1beta1` 제거.)

전체 정리

이번 실습에서는: 가벼운 정리

```
# 네임스페이스 통째로 삭제 → 안의 모든 리소스 사라짐
kubectl delete namespace ch4

# 클러스터는 그대로 살아있음 → 다음에 또 실습 가능
```

다시 시작할 때

```
kubectl create namespace ch4
kubectl config set-context --current --namespace=ch4
```

완전 정리 (발표 끝나면)

```
kind delete cluster --name ch4
osascript -e 'quit app "Docker"'
```

이번 실습으로 확인된 것 (회고)

주제	증명한 것
Liveness Probe	RESTARTS 카운터 증가 = "kubelet이 자동 재시작"의 직접 증거
ReplicationController	Pod 1개 죽이면 즉시 새로 채움. 라벨 매칭만으로 관리 대상 결정. RC가 살아있는 한 Pod 죽이는 게 의미 없음 → reconciliation 루프
ReplicaSet	apiVersion <code>apps/v1</code> , <code>matchLabels</code> . 동작은 RC와 동일
DaemonSet	노드 라벨 추가 → 즉시 Pod 생성 / 라벨 제거 → 즉시 Pod 제거. 노드당 정확히 1개 의 시각적 확인 (kind 멀티노드 덕)
Job	Completed 상태로 Pod 보존 → 결과/로그 사후 확인 가능. 병렬 처리 (parallelism)
CronJob	schedule + jobTemplate. 수동 트리거(<code>kubectl create job --from=cronjob/...</code>)로 즉시 실행

발표 한 줄 요약

"Pod을 직접 만들지 말고, 컨트롤러를 통해 관리하라."